

Assignment 2 (Weeks 3 and 4) — Solutions

Part I: Short Answer / Reasoning Questions

Q1. NULL Comparison and Aggregation

The query `SELECT COUNT(*) FROM Student WHERE phone_number = NULL;` always returns 0 because in SQL, NULL represents an unknown or missing value. Any comparison with NULL using `=`, `!=`, `<`, `>`, etc., evaluates to **UNKNOWN** (not TRUE or FALSE). Since the WHERE clause only passes rows that evaluate to TRUE, no rows ever satisfy this condition.

Corrected query:

```
SELECT COUNT(*) FROM Student WHERE phone_number IS NULL;
```

Why NULL is excluded from aggregates and comparisons: NULL means "value is unknown." Including an unknown value in a sum, average, or count would produce a meaningless result. For example, `AVG(salary)` skips NULLs because averaging a known number with an unknown one yields an unknown. Similarly, comparisons like `5 > NULL` cannot be resolved to TRUE or FALSE, so SQL uses three-valued logic (TRUE, FALSE, UNKNOWN) and excludes UNKNOWN rows from results.

Q2. DISTINCT with GROUP BY

Yes, the student is correct — the DISTINCT keyword is redundant here. The query groups rows by department, so each group corresponds to exactly one unique department value. The result set already has one row per department. Applying DISTINCT on top of this does nothing because there are no duplicate (department, COUNT(*)) combinations possible when department is the grouping column. DISTINCT would only matter if a non-grouped column appeared in SELECT (which would itself be invalid in standard SQL) or if the GROUP BY did not fully determine uniqueness of the selected columns.

Q3. INNER JOIN vs LEFT JOIN Row Count Difference

If the LEFT JOIN returns more rows than the INNER JOIN, it tells us that there exist customers in the Customer table who have no matching rows in the Order table — i.e., some customers have never placed an order. An INNER JOIN only returns rows where a match exists in both tables, so customers without orders are dropped. A LEFT JOIN preserves every row from the left table (Customer), filling in NULLs for the Order columns where no match exists. The extra rows in the LEFT JOIN result represent exactly those customers with zero orders.

Q4. Invalid Non-Aggregated Column in GROUP BY

The query is invalid because `employee_name` appears in the SELECT list but is neither part of the GROUP BY clause nor wrapped in an aggregate function. When we GROUP BY department, each result row represents an entire group of employees in that department. The `AVG(salary)` is well-defined (one average per group), and department is the grouping key. But `employee_name` could have many different values within a single department group. If SQL allowed this, it would have to arbitrarily pick one `employee_name` from the group, which would be misleading and non-deterministic. Standard SQL therefore rejects such queries.

Corrected query:

```
SELECT department, AVG(salary)
FROM Employee
GROUP BY department;
```

(If employee_name is also needed, it must be added to the GROUP BY clause — which changes the grouping semantics — or wrapped in an aggregate like MIN(employee_name) or MAX(employee_name).)

Q5. WHERE vs HAVING for Aggregate Conditions

The error is that AVG(salary) is used inside the WHERE clause. In SQL's logical execution order, WHERE is evaluated **before** GROUP BY and aggregation. At the time WHERE filters individual rows, the groups have not yet been formed, so aggregate functions like AVG() have no meaning. The HAVING clause, which executes **after** GROUP BY and aggregation, is the correct place to filter based on aggregate values.

Corrected query:

```
SELECT department, AVG(salary)
FROM Employee
GROUP BY department
HAVING AVG(salary) > 80000;
```

Q6. Correlated vs Non-Correlated Subqueries

(a) Products priced above overall average: A **non-correlated** subquery is appropriate. The overall average price is a single fixed value that does not depend on any particular row of the outer query. The subquery SELECT AVG(price) FROM Product is evaluated once, and the result is reused for every row in the outer query.

(b) Employees earning more than their department's average: A **correlated** subquery is appropriate. The average salary depends on the department of the current outer row. The subquery must reference the outer table's department value, so it is re-evaluated for each row of the outer query: SELECT AVG(salary) FROM Employee e2 WHERE e2.department = e1.department.

Performance implication: A correlated subquery is re-executed once for every row in the outer table. If the outer table has N rows, the subquery runs N times, leading to potentially $O(N \times M)$ cost (where M is the inner table size). A non-correlated subquery runs only once and its result is cached. For large outer tables, correlated subqueries can be significantly slower unless the optimizer can rewrite them as joins.

Part II: Long Answer Question (SQL Queries)

(a) Customers who have never placed an order

```
SELECT c.customer_id, c.name, c.email, c.phone
FROM Customer c
LEFT JOIN `Order` o ON c.customer_id = o.customer_id
WHERE o.order_id IS NULL;
```

Justification: A LEFT JOIN preserves all customers, including those with no matching orders. The WHERE o.order_id IS NULL filter then isolates only those customers whose join produced no match — i.e., customers who have never placed an order. An INNER JOIN would discard these customers entirely. An equivalent approach using NOT EXISTS or NOT IN (subquery) would also work, but the LEFT JOIN + IS NULL pattern is widely readable and typically well-optimized.

(b) Average order value per restaurant (more than 5 orders)

```
SELECT o.restaurant_id, AVG(order_total) AS avg_order_value
FROM `Order` o
INNER JOIN (
    SELECT oi.order_id, SUM(mi.price * oi.quantity) AS order_total
    FROM OrderItem oi
    INNER JOIN MenuItem mi ON oi.item_id = mi.item_id
    GROUP BY oi.order_id
) ov ON o.order_id = ov.order_id
GROUP BY o.restaurant_id
HAVING COUNT(DISTINCT o.order_id) > 5;
```

Justification: The inner subquery computes each order's total value by joining OrderItem with MenuItem and summing price × quantity per order. The outer query then joins this with the Order table to group by restaurant and compute the average of these per-order totals. HAVING COUNT(DISTINCT o.order_id) > 5 ensures we only include restaurants with more than 5 orders. INNER JOINS are used because we only want orders that have items and valid menu items.

(c) Restaurants with above-platform-average order value

```
SELECT restaurant_id, avg_order_value
FROM (
    SELECT o.restaurant_id, AVG(order_total) AS avg_order_value
    FROM `Order` o
    INNER JOIN (
        SELECT oi.order_id, SUM(mi.price * oi.quantity) AS order_total
        FROM OrderItem oi
        INNER JOIN MenuItem mi ON oi.item_id = mi.item_id
        GROUP BY oi.order_id
    ) ov ON o.order_id = ov.order_id
    GROUP BY o.restaurant_id
    HAVING COUNT(DISTINCT o.order_id) > 5
) rest_avg
WHERE avg_order_value > (
    SELECT AVG(order_total)
    FROM (
        SELECT oi.order_id, SUM(mi.price * oi.quantity) AS order_total
        FROM OrderItem oi
```

```

        INNER JOIN MenuItem mi ON oi.item_id = mi.item_id
        GROUP BY oi.order_id
    ) all_orders
);

```

Justification: A **non-correlated** subquery is used to compute the platform-wide average order value because this value is a single scalar that does not depend on any particular restaurant. The outer query wraps the result from part (b) and filters restaurants whose average exceeds the platform average. A correlated subquery is unnecessary here since the platform-wide average is the same for every restaurant row — computing it once is both simpler and more efficient.

(d) Most ordered menu item per restaurant

```

SELECT restaurant_id, item_id, item_name, total_qty
FROM (
    SELECT mi.restaurant_id, mi.item_id, mi.item_name,
           SUM(oi.quantity) AS total_qty,
           RANK() OVER (PARTITION BY mi.restaurant_id ORDER BY SUM(oi.quantity)
                        DESC) AS rnk
    FROM OrderItem oi
    INNER JOIN MenuItem mi ON oi.item_id = mi.item_id
    GROUP BY mi.restaurant_id, mi.item_id, mi.item_name
) ranked
WHERE rnk = 1;

```

Justification: A window function (RANK()) partitioned by restaurant cleanly solves the "top-N per group" problem. The INNER JOIN between OrderItem and MenuItem is correct because we only want items that have actually been ordered.

Limitation of GROUP BY + MAX: A naive approach like grouping by restaurant and selecting MAX(SUM(quantity)) does not work because you cannot nest aggregate functions. Even if you compute the max total quantity per restaurant, you cannot directly retrieve the *item_id* or *item_name* that achieved that maximum within the same GROUP BY query without a self-join or subquery. GROUP BY + MAX tells you *what* the maximum value is, but not *which row* produced it. Window functions like RANK() or ROW_NUMBER() elegantly solve this by assigning ranks within each partition.

Part III: Normalization

Q1. Functional Dependencies

Based on the given assumptions, the functional dependencies (FDs) are:

1. **student_id** → **student_name** (each student has a unique name)
2. **course_id** → **course_name, instructor_id, instructor_name, instructor_office** (each course has exactly one instructor, and the instructor's details follow from the course)
3. **instructor_id** → **instructor_name, instructor_office** (each instructor has one name and one office)
4. **(student_id, course_id)** → **grade, enrollment_date** (the composite key determines grade and enrollment date)

Q2. Is the table in 1NF?

Yes, the table is in 1NF. Every column contains atomic (indivisible) values — there are no repeating groups, multi-valued attributes, or nested structures. Each cell holds a single value, and the table has a valid primary key (student_id, course_id) that uniquely identifies each row.

Q3. Partial Dependency Violating 2NF

The composite primary key is (student_id, course_id). A **partial dependency** exists when a non-key attribute depends on only *part* of the composite key. Here, several partial dependencies exist:

- **student_id** → **student_name** — student_name depends only on student_id, not the full key.
- **course_id** → **course_name, instructor_id, instructor_name, instructor_office** — these depend only on course_id.

Anomaly example (Update Anomaly): If instructor "Dr. Sharma" changes their office from "Room 301" to "Room 405", we must update instructor_office in every row where that instructor appears. If we miss even one row, the table becomes inconsistent — the same instructor would show two different offices. Similarly, an **insertion anomaly** occurs if we want to add a new course with its instructor but cannot do so unless a student enrolls (since student_id is part of the primary key). A **deletion anomaly** occurs if the last student enrolled in a course drops it — we lose all information about the course and its instructor.

Q4. Transitive Dependency Violating 3NF

A transitive dependency exists: **course_id** → **instructor_id** → **instructor_name, instructor_office**. The attributes instructor_name and instructor_office depend on instructor_id, which in turn depends on course_id (part of the primary key). These attributes are transitively dependent on the key rather than directly dependent on it.

Anomaly: If an instructor's office changes, it must be updated in every row containing that instructor. If an instructor is removed from all courses, their name and office information is lost entirely (deletion anomaly). If we want to store a new instructor who hasn't been assigned a course yet, we cannot insert them (insertion anomaly).

Q5. Decomposition into 3NF

The decomposed relations in 3NF are:

1. Student(student_id, student_name)

Justified by: $\text{student_id} \rightarrow \text{student_name}$. Separates student information so that student names are stored once.

2. Instructor(instructor_id, instructor_name, instructor_office)

Justified by: $\text{instructor_id} \rightarrow \text{instructor_name}, \text{instructor_office}$. Eliminates the transitive dependency. Instructor details are stored in one place.

3. Course(course_id, course_name, instructor_id)

Justified by: $\text{course_id} \rightarrow \text{course_name}, \text{instructor_id}$. Each course maps to one instructor via a foreign key. Eliminates partial dependency on course_id .

4. Enrollment(student_id, course_id, grade, enrollment_date)

Justified by: $(\text{student_id}, \text{course_id}) \rightarrow \text{grade}, \text{enrollment_date}$. This is the associative entity capturing the many-to-many relationship between students and courses, with only fully dependent attributes.

Part IV: Design Justification

In Part II(a), finding customers who have never placed an order could be written using either a LEFT JOIN with an IS NULL check or a NOT EXISTS / NOT IN subquery. I chose the LEFT JOIN approach because it is highly readable and makes the intent clear: preserve all customers, then filter for those with no matching order. A NOT IN subquery can produce incorrect results if the subquery returns NULLs (since IN comparisons with NULL yield UNKNOWN, potentially excluding valid rows). NOT EXISTS avoids this pitfall and is semantically equivalent, but the LEFT JOIN pattern is more intuitive for most readers and is well-optimized by modern query planners. Performance-wise, both approaches are comparable on indexed foreign keys, but the LEFT JOIN avoids the subtle NULL trap of NOT IN.

Regarding normalization in Part III, splitting the Enrollment table into 3NF eliminates redundancy and anomalies, but it trades off query convenience. To retrieve a student's enrollment along with course name, instructor name, and office, we now need a multi-table JOIN across four tables instead of a single-table SELECT. This increases query complexity and can impact performance due to the cost of joining. In real systems, intentional denormalization is sometimes used — for example, keeping instructor_name in the Course table or maintaining a materialized view — to speed up frequently-run read-heavy queries, accepting the trade-off of managing redundancy through application logic or triggers. Data warehouses and reporting systems commonly favor denormalized schemas (star/snowflake) precisely because read performance outweighs the cost of controlled redundancy.